



Trabajo Final M5:

Juego Pong entre 2 jugadores remoto

INTERACCIÓN, SENSORES Y TRANSDUCTORES

Marc Forés Doménech
Carlos Clariana Botella

Tecnología Digital y Multimedia | 28-12-2025

Contenido

1.	Introducción	2
2.	Objetivos	2
3.	Desarrollo.....	3
3.1	Arquitectura general del sistema.....	3
3.2	Comunicación wifi y protocolo de intercambio de datos.....	3
3.3	Control de palas mediante sensores imu	4
3.4	Algoritmo de movimiento y rebote de la pelota	4
3.5	Representación gráfica del juego	4
3.6	Algoritmo general del sistema (pseudocódigo)	5
4.	Conclusiones.....	6

1. Introducción

El presente trabajo consiste en el desarrollo de un **juego Pong para dos jugadores remotos**, implementado sobre dos dispositivos **M5Stack-FIRE** y programado en **MicroPython**. El objetivo principal es demostrar la integración de distintos conceptos vistos en la asignatura, como el uso de sensores, la comunicación en red, el diseño de algoritmos y la interacción en tiempo real entre dispositivos empotrados.

El clásico juego Pong ha sido elegido como base del proyecto por tratarse de un ejemplo sencillo desde el punto de vista conceptual, pero lo suficientemente completo como para abordar múltiples aspectos técnicos relevantes. En este caso, el juego se ha ampliado para funcionar de manera **distribuida**, permitiendo que dos jugadores situados en dispositivos diferentes interactúen entre sí a través de una red WiFi.

Cada jugador controla una pala mediante el **movimiento físico del dispositivo**, utilizando el acelerómetro y giroscopio integrados en el M5Stack-FIRE. Esta forma de control proporciona una interacción más natural e intuitiva, y permite trabajar con el procesado de señales procedentes de sensores reales. La posición de las palas y de la pelota se sincroniza entre ambos dispositivos mediante el intercambio de mensajes en red. El sistema se ha diseñado siguiendo una arquitectura **maestro-esclavo**, donde uno de los dispositivos centraliza la lógica del juego y el otro actúa como terminal remoto. Esta decisión simplifica la sincronización del estado del juego y reduce posibles inconsistencias derivadas de la comunicación inalámbrica.

2. Objetivos

Los objetivos base del presente trabajo práctico hacen alusión a los siguientes apartados:

- Desarrollar un juego Pong funcional para dos jugadores remotos.
- Utilizar dos dispositivos M5Stack-FIRE comunicados mediante WiFi.
- Implementar el control de las palas usando el acelerómetro/giroscopio (IMU), así como diseñar una arquitectura maestro-esclavo clara y eficiente.
- Implementar un algoritmo de movimiento y rebote de la pelota.
- Sincronizar el estado del juego mediante mensajes UDP en formato JSON.
- Conseguir una ejecución fluida y estable en tiempo real.
- Validar el funcionamiento mediante vídeos demostrativos de las conexiones y resultados obtenidos.

3. Desarrollo

Cada subapartado se ha implementado de forma modular, con lectura de datos, análisis, cálculo y visualización. A continuación, se describe cada una con su correspondiente algoritmo en pseudocódigo para concluir el desarrollo.

3.1 ARQUITECTURA GENERAL DEL SISTEMA

El sistema desarrollado se basa en una arquitectura claramente diferenciada de tipo **maestro-esclavo**, en la que cada dispositivo M5Stack-FIRE asume un rol específico dentro del juego. Esta decisión de diseño se ha tomado con el objetivo de simplificar la sincronización del estado del juego y evitar problemas derivados de cálculos duplicados o inconsistencias entre ambos dispositivos.

El **maestro**, correspondiente al jugador 1, actúa como nodo central del sistema. En él se concentra toda la lógica principal del juego: cálculo del movimiento de la pelota, detección de colisiones, gestión del marcador y envío del estado global al otro dispositivo. Por otro lado, el **esclavo**, correspondiente al jugador 2, tiene un papel más ligero, centrado en la lectura de la IMU, el envío de la posición de su pala al maestro y la representación gráfica del estado del juego recibido.

Esta separación de responsabilidades permite que el sistema funcione de manera estable incluso en presencia de pequeñas pérdidas de paquetes o retardos en la comunicación, ya que solo existe una fuente de verdad para el estado del juego.

3.2 COMUNICACIÓN WIFI Y PROTOCOLO DE INTERCAMBIO DE DATOS

La comunicación entre ambos dispositivos se realiza a través de una red **WiFi**, utilizando sockets **UDP**. Se ha optado por este protocolo debido a su baja latencia y simplicidad, características especialmente importantes en aplicaciones interactivas en tiempo real como los videojuegos. Aunque UDP no garantiza la entrega de los paquetes, en este contexto resulta suficiente, ya que el estado del juego se actualiza de forma continua.

Los datos intercambiados se codifican en formato **JSON**, lo que facilita su lectura, depuración y extensión futura. El esclavo envía periódicamente mensajes que contienen la posición de su pala, mientras que el maestro responde con mensajes que incluyen el estado completo del juego: posiciones de ambas palas, posición de la pelota y puntuaciones de los jugadores.

Para evitar que el esclavo represente estados antiguos debido a la llegada de paquetes fuera de orden, el maestro incluye en cada mensaje un **número de secuencia**. El esclavo compara este valor con el último recibido y descarta cualquier mensaje que no sea más reciente, garantizando así una representación coherente del juego.

3.3 CONTROL DE PALAS MEDIANTE SENSORES IMU

El control de las palas se realiza utilizando la **unidad de medida inercial (IMU)** integrada en los dispositivos M5Stack-FIRE. A partir de los valores del acelerómetro se calcula el ángulo de inclinación del dispositivo (pitch), que se traduce directamente en un desplazamiento vertical de la pala en pantalla.

Antes de comenzar el juego, se lleva a cabo un proceso de **calibración**, durante el cual se promedia la lectura del sensor con el dispositivo en reposo. Este valor de referencia se utiliza posteriormente para eliminar posibles desviaciones iniciales. Para mejorar la calidad del control, las lecturas del sensor se filtran mediante un **filtro paso bajo**, que suaviza los movimientos y reduce el ruido.

Además, se implementa una **zona muerta**, de manera que pequeñas inclinaciones o vibraciones no provoquen movimientos indeseados de la pala. El resultado es un sistema de control suave, estable y preciso, que mejora notablemente la experiencia de juego.

3.4 ALGORITMO DE MOVIMIENTO Y REBOTE DE LA PELOTA

El algoritmo que gobierna el movimiento de la pelota se ejecuta exclusivamente en el dispositivo maestro. En cada iteración del bucle principal, la posición de la pelota se actualiza en función de su velocidad y del tiempo transcurrido desde la última actualización.

Se comprueba de forma continua la colisión de la pelota con los bordes superior e inferior de la pantalla, invirtiendo la velocidad vertical cuando se produce un impacto. De igual forma, se detecta la colisión con las palas de ambos jugadores. En este caso, además de invertir la velocidad horizontal, se modifica la velocidad vertical en función del punto de impacto, calculado respecto al centro de la pala. Este comportamiento introduce variabilidad en la trayectoria de la pelota y hace el juego más dinámico.

Cuando la pelota sale completamente por uno de los laterales de la pantalla, se considera que se ha producido un punto. El marcador se actualiza en consecuencia y la pelota se reinicia desde el centro de la pantalla con una dirección inicial adecuada.

3.5 REPRESENTACIÓN GRÁFICA DEL JUEGO

La representación gráfica se realiza directamente sobre la pantalla LCD del M5Stack-FIRE, utilizando primitivas gráficas sencillas como rectángulos y líneas. Las palas y la pelota se dibujan como rectángulos sólidos, mientras que la red central y el marcador se muestran de forma estática o semiestática.

Para optimizar el rendimiento y evitar parpadeos, en cada iteración se borra la posición anterior de los elementos móviles antes de dibujar la nueva. Este enfoque resulta eficiente y adecuado para las limitaciones de hardware del dispositivo, permitiendo una tasa de refresco fluida.

3.6 ALGORITMO GENERAL DEL SISTEMA (PSEUDOCÓDIGO)

El funcionamiento global del sistema puede resumirse mediante el siguiente pseudocódigo:

Maestro:

Inicializar WiFi e IMU

Calibrar sensores

Inicializar estado del juego

Mientras el juego esté activo:

 Recibir posición de la pala del esclavo

 Leer IMU del maestro

 Actualizar posición de la pala del jugador 1

 Calcular físicas y colisiones de la pelota

 Actualizar marcador si es necesario

 Dibujar el estado del juego

 Enviar estado completo al esclavo

Esclavo:

Inicializar WiFi e IMU

Calibrar sensores

Mientras el juego esté activo:

 Leer IMU del esclavo

 Actualizar posición de la pala del jugador 2

 Enviar posición al maestro

 Recibir estado del juego

 Dibujar palas, pelota y marcador

4. Conclusiones

A lo largo de este proyecto se ha desarrollado con éxito un juego Pong multijugador remoto sobre dispositivos M5Stack-FIRE, cumpliendo con todos los objetivos planteados inicialmente. El sistema permite la interacción en tiempo real entre dos jugadores, haciendo uso de sensores físicos y comunicación inalámbrica.

El trabajo realizado ha permitido consolidar conocimientos relacionados con la programación de sistemas empujados, el uso de sensores IMU, la comunicación en red mediante UDP y el diseño de algoritmos de control y sincronización. La arquitectura maestro-esclavo ha demostrado ser una solución adecuada para este tipo de aplicaciones, ya que simplifica la gestión del estado global del juego y reduce problemas de incoherencia entre dispositivos.

Asimismo, el uso del acelerómetro como método de control aporta una experiencia de usuario más intuitiva y diferente a la de los controles tradicionales, demostrando el potencial de este tipo de sensores en aplicaciones interactivas.

Como posibles líneas de mejora, se podría optimizar la gestión de la latencia mediante técnicas de predicción o interpolación, añadir efectos sonoros y visuales, o ampliar el juego con menús y opciones de configuración. No obstante, el resultado obtenido es plenamente satisfactorio y constituye una base sólida para futuros desarrollos más complejos en el ámbito de los sistemas empujados y las aplicaciones interactivas en red.